

XRComm FCS Wideband Receive Platform

Quick Start & User Guide

XRComm Inc.

June 2026

XRComm FCS Wideband Receive Platform

Quick Start & User Guide

Platform	XRComm FCS Wideband Receive Platform
Driver Version	v1.4.3
Variant	Wideband
Audience	End users and system administrators
Classification	Proprietary

© 2026 XRComm Inc. – All Rights Reserved.

Contents

1 Terminology	3
2 System Topology	4
2.1 Waveform Playback System Topology	4
3 Package Contents	4
4 Installation	5
4.1 Extract the Package	5
4.2 Run the Installer	5
4.3 Uninstallation	6
5 Verifying the Services	6
6 Controlling the Platform over gRPC	6
6.1 Endpoint	6
6.2 Client Setup	6
6.3 Control Commands	7
6.4 Telemetry Fields	7
6.5 Mode Rules	8
7 Recording Captures and Reading Them Back	8
7.1 Automatic Read-Back Configuration	8
7.2 Setting Read-Back Parameters via gRPC	8
7.3 Recording Workflow	8
7.4 Output Files	8
7.5 Optional Offline NVMe Retrieval	9
8 Waveform Playback Platform	9
8.1 Playback Directory Structure	9
8.2 Playback Configuration Keys (inputs/config.ini)	10
8.3 Running the Playback Control Server	10
8.4 Running the Playback Client (Client Machine)	10
8.4.1 Playback gRPC Client Commands	11
9 Appendix A: Environment Prerequisites	11
9.1 Dependency Version Check Table	11
9.2 Hugepages	11
9.3 Device Binding (DPDK / SPDK)	12
9.4 Privileges	12

1

Terminology

The following terms are used throughout this document.

Term	Meaning
FlexServer	The physical host machine running the XRComm platform driver and gRPC control service. All platform binaries execute here.
Client Machine	Any machine (including the FlexServer itself) from which you run the Python gRPC control client to operate the platform.
Platform Driver	The <code>xrcomm-server</code> binary. Receives the high-speed RF stream from the NIC, manages shared memory, records to NVMe, and dispatches data to customer applications.
Control Service	The <code>xrcomm-grpc-ctrl</code> binary. Provides a gRPC interface (TCP port 50051) for remotely controlling the Platform Driver.
Customer Application	Your own software that connects to the Platform Driver via shared memory to consume IQ samples or full packets.
IQ Delivery	Mode in which interleaved int16 I/Q sample batches are dispatched to connected applications via zero-copy shared memory.
Full-Packet Delivery	Mode in which raw network packets are forwarded to a single application using DPDK secondary process attachment. Requires <code>sudo</code> .
NVMe Recording	Mode in which the received RF stream is written directly to the NVMe drive via SPDK for later offline extraction.

2

System Topology

The platform operates as a set of background services on the FlexServer. Remote or local clients can control the system over the network using a Python gRPC client.

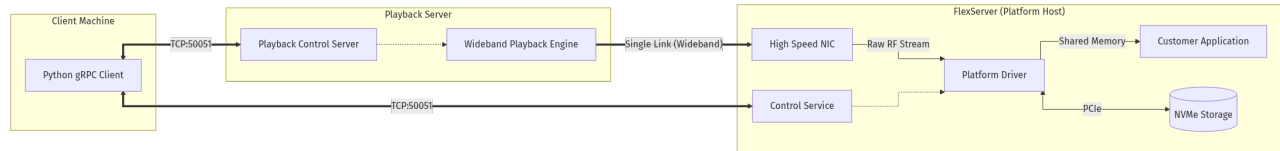


Figure 1: XRComm FCS Wideband Receive Platform Topology

Data flow:

1. The **High Speed NIC** receives the raw RF stream and delivers it to the **Platform Driver** via DPDK.
2. The **Platform Driver** dispatches IQ samples to connected **Customer Applications** via zero-copy shared memory.
3. Simultaneously, the Platform Driver can record the stream to the **NVMe Storage** via SPDK.
4. The **Control Service** communicates with the Platform Driver over IPC, allowing remote management.
5. The **Python gRPC Client** (on the **Client Machine** or on the FlexServer itself) connects to the Control Service over TCP port 50051.

2.1

Waveform Playback System Topology

The playback system acts as a signal source, cabled directly to the receive ports of the FlexServer.

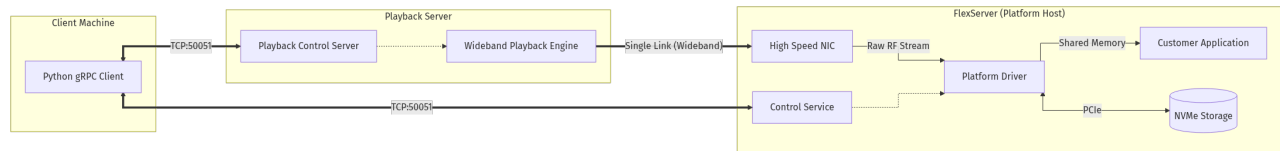


Figure 2: XRComm FCS Wideband Playback Topology

[!IMPORTANT] Playback Hardware Separation: The Playback Engine (`xrcomm_playback`) operates as a high-rate transmitter (pacing IQ samples to DPDK at wire-rate). To prevent host resource contention (such as CPU context switches, PCIe bus saturation, or memory channel exhaustion) with the platform receiver on the FlexServer, **the Playback Engine and its Control Server must run on a separate server machine**. Do not execute the playback engine on the FlexServer in production.

3

Package Contents

After extracting the tarball, you will find the following directory structure:

```
XRComm_FCS_platform/
  install.sh           # Automated installer
  uninstall.sh         # Clean uninstaller
  LICENSE              # Software license agreement
  VERSION              # Release version identifier
  CHANGELOG.md         # Release notes
  Wideband_Quickstart.pdf # This document

XRComm_FCS_wideband_platform/
  config/
```

```

    read_config.ini          # Wideband auto read-back config
XRComm_platform_drivers/
  bin/                      # Precompiled platform driver binary
  include/                  # Public C headers for app development
  service/                  # Systemd unit file templates
XRComm_platform_gRPC/
  bin/                      # Precompiled gRPC control service binary
  client/                   # Python gRPC client scripts
  proto/                    # Protobuf service definition (.proto)
  service/                  # Systemd unit file templates

XRComm_wideband_hello_world/
  hello_world.c             # IQ delivery application (no sudo)
  hello_world_fullpkt.c     # Full-packet delivery application (sudo)
  CMakeLists.txt            # Build configuration
  bin/                      # Output directory for compiled applications

XRComm_wideband_playback/
  bin/                      # Precompiled playback engine and control server
  inputs/                   # Playback config and waveform files
  grpc/                     # gRPC client and proto files

XRComm_NVMe_loader/
  bin/                      # Precompiled NVMe reader binary
  config/                   # Optional offline reader defaults
  scripts/                  # Optional offline extraction (read_nvme.sh)

```

4

Installation

4.1

Extract the Package

Transfer the tarball to your FlexServer and extract it:

```

tar -xzf XRComm_FCS_platform.tar.gz
cd XRComm_FCS_platform

```

4.2

Run the Installer

```
sudo ./install.sh
```

When prompted, accept the license agreement and select **Option 1** for the Wideband Platform.

The installer performs the following actions automatically:

Step	What it does
Dependency installation	Installs python3-grpcio, python3-protobuf, and python3-grpc-tools via apt.
Systemd service creation	Creates and enables xrcomm-server.service and xrcomm-grpc-ctrl.service.
Header file mapping	Symlinks the public C headers into /usr/include/xrcomm-wideband/.

Step	What it does
Proto compilation	Generates the Python gRPC stubs (pipeline_control_pb2.py) in the client directory.
Service startup	Starts both services immediately.

4.3

Uninstallation

```
sudo ./uninstall.sh
```

This stops and disables the systemd services, removes the header symlinks, and uninstalls the gRPC Python packages.

5

Verifying the Services

After installation, both platform services run in the background. Verify their status:

```
systemctl status xrcomm-server.service
systemctl status xrcomm-grpc-ctrl.service
```

Both should report active (running).

To inspect live logs:

```
journalctl -u xrcomm-server.service -f      # Platform Driver logs
journalctl -u xrcomm-grpc-ctrl.service -f    # Control Service logs
```

To restart the services after a configuration change:

```
sudo systemctl restart xrcomm-server.service
sudo systemctl restart xrcomm-grpc-ctrl.service
```

6

Controlling the Platform over gRPC

The remote control client is designed to run from the **Client Machine**.

6.1

Endpoint

The Control Service listens on 0.0.0.0:50051 by default. It starts automatically as a systemd service and does not need to be launched manually.

6.2

Client Setup

The Python gRPC client can be run from **any machine** (specifically the **Client Machine**) that has network access to the FlexServer:

- **From the FlexServer itself** (via loopback 127.0.0.1) for internal verification during installation.
- **From a remote Client Machine** for standard operational use.

Install the Python dependencies once per machine:

```
cd XRComm_FCS_wideband_platform/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
```

The installer already generates the protobuf Python bindings on the FlexServer. If you are setting up the client on a separate machine, copy the `client/` directory there and generate the stubs:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
  --grpc_python_out=. pipeline_control.proto
```

6.3

Control Commands

Usage syntax:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 <command> [value]
```

Replace `<server-ip>` with the IP address of your FlexServer (use `127.0.0.1` when running locally).

Command	What it does
<code>get-flags</code>	Show current delivery and recording mode flags.
<code>set-ip on off</code>	Turn IQ-sample delivery to applications on or off.
<code>set-dsp on off</code>	Turn full-packet delivery to applications on or off.
<code>set-logging on off</code>	Start or stop NVMe recording. Stopping triggers automatic read-back (Section 7).
<code>set-read-capture <seconds></code>	Set automatic read-back duration, in seconds.
<code>set-read-rate <hz></code>	Set the receiver sample rate, in Hz, used to size the read-back.
<code>get-stats</code>	Show pipeline counters and platform telemetry.
<code>get-mode-stats</code>	Show received/processed/dropped counts per delivery mode and per application.
<code>list-ips</code>	List the applications currently connected to the platform.
<code>watch-status</code>	Stream live status once per second (Ctrl-C to stop).
<code>shutdown</code>	Gracefully shut the platform down.

6.4

Telemetry Fields

The `get-stats` and `watch-status` commands report platform telemetry:

Field	Meaning
<code>hsp_port</code>	High-speed receive port in use.
<code>samples_received</code>	Total samples received from the interface.
<code>samples_logged</code>	Samples written to NVMe (zero unless recording).
<code>samples_processed_secondary</code>	Samples consumed by your application(s).
<code>packets_dropped</code>	Receive-overflow and app-backlog drops, combined.

6.5

Mode Rules

- The three delivery/recording modes (IQ, full-packet, and recording) are independent and may be enabled in any combination.
- Full-packet delivery serves one application at a time. IQ delivery may be shared by multiple applications.

7

Recording Captures and Reading Them Back

7.1

Automatic Read-Back Configuration

Two values control the automatic read-back. They are set from the control client and stored in `XRComm_FCS_wideband_platform/config/read_config.ini`. The installer exports this absolute path as `XRCOMM_READ_CONFIG` for both platform services.

Key	Default	Meaning
<code>read_duration_seconds</code>	1.0	Seconds of the most recent capture to copy back when recording stops.
<code>sample_rate_hz</code>	800000000	Receiver sample rate (Hz), used to convert duration into data size.

Do not copy the 8T8R per-channel config into the Wideband platform; the schemas are different. The amount copied back is `seconds x sample_rate x 4 bytes` (each sample is one little-endian int16 I value followed by one little-endian int16 Q value), rounded to the nearest whole record and capped to the capture length. At 800 MHz, 1 second copies approximately 3,051.76 MiB.

7.2

Setting Read-Back Parameters via gRPC

The parameters are configured from the **Client Machine** running the gRPC client:

Set the read-back duration to 1.5 seconds

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-capture 1.5
```

Set the receiver sample rate to 800 MHz

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-rate 800000000
```

7.3

Recording Workflow

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging on      # Start
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging off   # Stop -> auto read-back
python3 xrcomm_grpc_client.py --host <server-ip>:50051 watch-status      # Confirm read-back progress
```

7.4

Output Files

Read-back files are written under the Platform Driver's working directory in `output_logs/readback_<timestamp>/:`

- `readback_<timestamp>_chunk_N.bin` – raw interleaved int16 little-endian IQ pairs (I, Q, I, Q, ...), split into 2 GB chunks.
- `readback_<timestamp>.idx` – a text manifest describing each block in the .bin files.

7.5

Optional Offline NVMe Retrieval

The offline NVMe reader is an optional tool for manual extraction after the platform has stopped. Use it to extract older or specific data blocks outside the automatic read-back workflow:

```
cd XRComm_NVMe_loader/scripts
sudo ./read_nvme.sh
```

The underlying binary (`xrcomm_nvme_reader`) also accepts options for partial extraction:

Option	Effect
<code>--first-mb=<N> / --first-gb=<N></code>	Extract the first N megabytes / gigabytes.
<code>--middle-mb=<N> / --middle-gb=<N></code>	Extract from the middle of the capture.
<code>--last-mb=<N> / --last-gb=<N></code>	Extract the last N megabytes / gigabytes.
<code>--flash</code>	Free the capture on the drive after successful full extraction.
<code>--force-flash</code>	Force delete even if extraction is partial or corrupted.

Extracted data uses the same format as the automatic read-back.

8

Waveform Playback Platform

The Wideband variant includes the Waveform Playback Engine (`XRComm_wideband_playback/`) for streaming pre-recorded IQ waveforms over the network interface.

8.1

Playback Directory Structure

The precompiled playback engine and control client files reside in the following directory layout:

```
XRComm_wideband_playback/
├── bin/
│   ├── xrcomm_grpc_ctrl          # Precompiled control server
│   └── xrcomm_playback          # Precompiled playback engine
├── grpc/
│   ├── client/
│   │   ├── README.md
│   │   ├── XRComm_wideband_playback_client.py # Playback client control
│   │   ├── pipeline_control_pb2.py          # Shipped precompiled stub
│   │   └── pipeline_control_pb2_grpc.py      # Shipped precompiled stub
│   └── proto/
│       └── pipeline_control.proto            # Service contract
└── inputs/
    ├── config.ini                      # Playback configuration
    └── playback_waveforms/
        ├── binary_search_tool.py        # Waveform helper
        ├── hexdump
        ├── split_bin.py                  # Stream splitter helper
        └── tone_512.bin                  # Sample waveform
```

8.2

Playback Configuration Keys (inputs/config.ini)

Start parameters are loaded from inputs/config.ini at startup.

Key	Default	Meaning
core_list	7,8,9	DPDK lcore IDs to assign to the playback engine (isolate these cores)
source_mode	1	1 = waveform file playback, 0 = streaming from NVMe drive capture
binary_filename	tone_512.bin	Waveform file resolved under inputs/playback_waveforms/
target_sample_rate_hz	800000000	Sample rate in Hz; 800000000 is 800 MHz
tx_port_id	0	Physical DPDK network port ID to transmit from
dest_mac	Site-specific	Destination MAC address for transmitted packets
loop_count	0	0 = infinite loops, N = stop after N loops
trigger_burst_size	1	Number of packets marked active when playback is started through gRPC

Reconfigure `dest_mac` before playback. Set it to the MAC address of the FlexServer receive NIC port that is physically cabled to the playback transmit port.

8.3

Running the Playback Control Server

Start the control server from the `XRComm_wideband_playback/` directory on the **Playback Server**. This gRPC control server is the only playback-side process customers need to launch manually; it drives the shipped playback binary.

```
export XRCOMM_PLAYBACK_ROOT=$(pwd)
./bin/xrcomm_grpc_ctrl 0.0.0.0:50051
```

8.4

Running the Playback Client (Client Machine)

Operate the playback remotely from the **Client Machine** using the precompiled Python stubs:

```
cd XRComm_wideband_playback/grpc/client
python3 XRComm_wideband_playback_client.py --endpoint <playback-server-ip>:50051 <command> [args]
```

8.4.1 Playback gRPC Client Commands

- `get-config` - Read the 8 current `config.ini` parameters.
- `get <field>` - Read a single configuration parameter.
- `set <field> <value>` - Set a single configuration parameter (takes effect next run).
- `start` - Launch the playback engine on the Playback Server.
- `stop` - Stop the running playback engine cleanly.
- `diag` - Return a single diagnostics snapshot (running time, transmit rate, etc.).
- `watch` - Stream live diagnostics once per second (Ctrl-C to stop).

9

Appendix A: Environment Prerequisites

These configurations are **prerequisites** for the FlexServer. They are typically set up once during initial server provisioning. If the platform fails to start or behaves unexpectedly, verify these settings first.

9.1

Dependency Version Check Table

Dependency	Required Version	Installation Command / Source
Ubuntu Linux	22.04 LTS or 24.04 LTS (64-bit)	Operating System standard install
DPDK	>= 23.11.1	<code>sudo apt install dpdk libdpdk-dev</code> ; docs: https://core.dpdk.org/doc/
SPDK	>= 24.01	Build from source; docs: https://spdk.io/doc/
gRPC & Protobuf	v1.62.0 (C++ runtime)	<code>sudo apt install libgrpc++-dev libprotobuf-dev</code>
Python 3	v3.10+	Operating System standard install
grpcio & tools	v1.62.0 (Python runtime)	<code>pip3 install grpcio grpcio-tools</code>
CMake	v3.16+	<code>sudo apt install cmake</code>
GCC	v11+	<code>sudo apt install build-essential</code>

9.2

Hugepages

File-backed hugepages are required for zero-copy memory pools:

```
sudo rm -rf /dev/hugepages/* /mnt/huge/*           # Clear stale allocations
sudo sysctl -w vm.nr_hugepages=0
sudo sysctl -w vm.nr_hugepages=112                # Adjust to your memory layout (112 x 2MB)
grep -i huge /proc/meminfo                         # Verify
```

9.3

Device Binding (DPDK / SPDK)

Before launching the server, identify the receive NIC interfaces and NVMe storage drive, then bind them to the correct DPDK/SPDK user-space drivers.

- **Network Interface Discovery:** View available NIC PCI addresses, kernel interface names, and MAC addresses before binding:

```
sudo dpdk-devbind.py --status
```

```
ip -br link
```

```
ethtool -P <interface-name>
```

Select the FlexServer receive NIC ports that are physically connected to the RF/playback source, then use their MAC addresses when configuring receive/playback peers.

- **Network Interface (DPDK Binding):** Bind the selected wideband receive NIC ports to the DPDK-compatible driver (vfio-pci) using their PCI addresses:

```
sudo modprobe vfio-pci
```

```
sudo dpdk-devbind.py --bind=vfio-pci <nic-pci-address> [<nic-pci-address> ...]
```

- **NVMe Drive (SPDK Binding):** Bind the recording NVMe PCIe drive to SPDK using the SPDK setup.sh script.

9.4

Privileges

- The Platform Driver and full-packet applications require `sudo` (hugepages and direct device access).
- IQ applications and the control client do not require elevated privileges.